Reinforcement Learning

An Introduction

Course Content

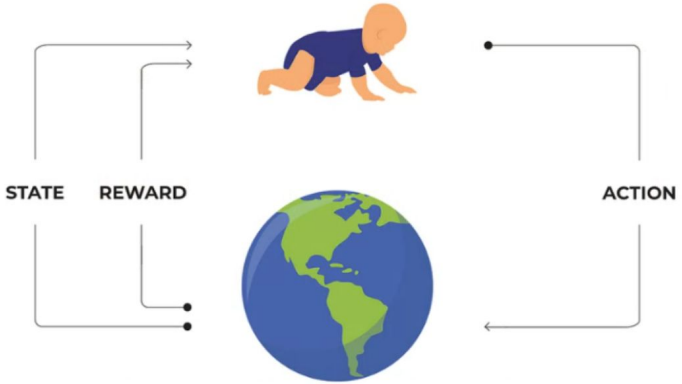
1. Introduction to Reinforcement Learning
2. Markov Decision Processes (MDPs)
3. Policy and Value Functions
4. Dynamic Programming (DP)
5. **Model-Free methods**
6. Value Function Approximation
7. Policy-Gradient and Actor-Critic Methods
8. Deep RL
9. TP Project

Recall …

Reinforcement Learning

Agent: The system that takes actions to be trained.

State: The information required by the agent to take an action. This info is observed from the environment.



Action: The decision or move that the agent makes at a particular state

Reward: Feedback received by the agent to evaluate the taken action under a certain state.

Environment: The external system with which the agent interacts.

Exploration & Exploitation

Exploration vs. Exploitation

- In RL, the agent faces a **dilemma** between:
  - **Exploration:** Trying **new actions** to discover valuable outcomes. (can be harmful...)
  - **Exploitation:** Choosing **actions** that have yielded **high rewards** in the past.

- **Goal:** Balance exploration and exploitation to maximize rewards over time.
- **Challenge:** Too much exploration can delay achieving rewards, while too much exploitation can lead to suboptimal long-term results.

Exploration: Discovering New Opportunities

- **Example 1 - A robot navigating a maze:**
  - The robot tries unfamiliar paths to locate shorter routes or more valuable rewards.

- **Example 2 - A recommendation system:**
  - Occasionally recommends new, lesser-known products to a user to learn their interests.

- **Benefit:** Exploration can uncover higher rewards that aren't immediately obvious.

Exploitation: Leveraging Known Information

- **Example 1 - A trading agent:**
  - Selects stocks it has previously identified as profitable, prioritizing consistency over discovering new options.

- **Example 2 - A game-playing AI:**
  - Repeats a high-reward move (e.g., a chess opening) that has led to victories in past games.

- **Benefit:** Exploitation capitalizes on known successes, ensuring steady rewards.

Model-Free methods

Model-Free Methods in RL

- **What are Model-Free Methods?**
  - Methods that **learn directly from interactions** with the environment.
  - **No explicit model** of the environment (transition probabilities or reward function) is required.

- **Key Characteristics:**
  - **Policy Evaluation:** Learn the value function under a given policy.
  - **Control:** Learn the optimal policy and its corresponding value function.
  - Can be applied to **large or unknown environments** where building a model is infeasible.

- **Applications:**
  - Robotics
  - Game AI
  - Autonomous Systems

Types of Model-Free Methods

1. **Monte Carlo (MC) Methods**
   - Learn from complete episodes of experience.
   - Value estimates are based on sample returns.
2. **Temporal-Difference (TD) Learning**
   - **Bootstrapping approach:** Updates value estimates using other estimated values.
   - **Example algorithms:**
     - TD(0)
     - SARSA
     - Q-Learning

Key Difference:

- TD methods can learn online, after every step.
- MC methods require complete episodes for updates.

Key notions for Today

- **Model-Free Methods**
- **Monte Carlo (MC)**
- **Temporal-Difference (TD)**
- **Bootstrapping**

Monte Carlo (MC) Methods

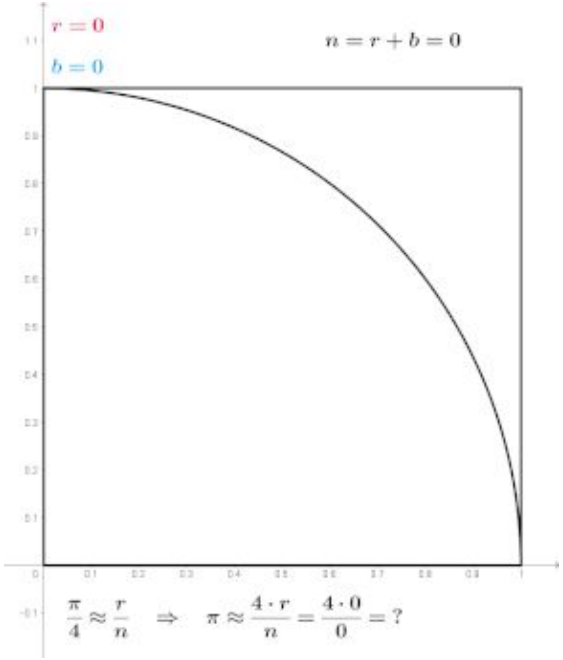
Introduction to Monte Carlo (MC)

What is Monte Carlo ?

- *In general* Monte Carlo methods estimate complex quantities by *statistical means*.
- Not specific to RL !

MC in RL :

- A model-free method for estimating value functions
- Relies on sampling from actual experiences (episodes) with the environment
- Used to evaluate and improve policies based on returns



MC for Prediction

Policy Evaluation

- **Objective:** Estimate v_π based on episodes generated following policy π

$$S_1, A_1, R_2, \ldots, S_k \sim \pi$$

- Remember, the return is the cumulative discounted reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-1} R_T$$

- The value function represents the expected return:

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right]$$

- Monte Carlo policy evaluation replaces the **expected return** with the **average of observed returns**.

First-Visit MC for Prediction

Policy Evaluation

- To estimate the value of state ***s***:
  a. For the **first time**-step ***t*** when state ***s*** is visited in an episode,
  b. Increment the counter ***N(s) = N(s) + 1*** ,
  c. add the return ***G(t)*** to the cumulative sum ***S(s) = S(s) + G(t)***.
- The estimated value is the average return:

$$V(s) = \frac{S(s)}{N(s)}$$

- According to the law of large numbers, as ***N(s) → ∞***, ***V(s) →*** $v_\pi(s)$

Every-Visit MC for Prediction

Policy Evaluation

- To estimate the value of state ***s***:
  a. For **every time**-step ***t*** when state ***s*** is visited in an episode,
  b. Increment the counter ***N(s) = N(s) + 1*** ,
  c. add the return ***G(t)*** to the cumulative sum ***S(s) = S(s) + G(t)***.
- The estimated value is the average return:

$$V(s) = \frac{S(s)}{N(s)}$$

- According to the law of large numbers, as ***N(s)*** → ∞, ***V(s)*** → $v_\pi(s)$

Incremental Mean

Proof

The mean $\mu_1, \mu_2, \ldots$ of a sequence $x_1, x_2, \ldots$ can be computed incrementally,

$$
\begin{aligned}
\mu_k &= \frac{1}{k} \sum_{j=1}^{k} x_j \\
&= \frac{1}{k} \left(x_k + \sum_{j=1}^{k-1} x_j \right) \\
&= \frac{1}{k} \left(x_k + (k-1)\mu_{k-1} \right) \\
&= \mu_{k-1} + \frac{1}{k} \left(x_k - \mu_{k-1} \right)
\end{aligned}
$$

Incremental Monte-Carlo Updates

- Update ***V(s)*** incrementally after an episode $S_1, A_1, R_2, \ldots, R_T$

- For each state ***S(t)*** with return ***G(t)***:

$$N(S_t) \leftarrow N(S_t) + 1$$

$$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}(G_t - V(S_t))$$

- In non-stationary problems, it can be beneficial to maintain a running average by gradually forgetting older episodes:

$$\boxed{V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))}$$

MC for Action Values

Policy Optimization

- Recall the action-value function is represented as:

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right]$$

In MC, $q_\pi(s, a)$ is estimated by **averaging** the observed returns ***G(t)*** from all episodes where ***S(t) = s*** and ***A(t) = a***:

$$Q(s, a) \approx \frac{1}{N(s, a)} \sum_{i=1}^{N(s,a)} G_t^{(i)} q_\pi(s, a)$$

where ***N(s, a)*** is the number of times action ***a*** was taken in state ***s***, and $G_t^{(i)}$ is the observed return from the i-th occurrence.

Importance Sampling

- Data may be collected under one policy ***b*** and used to estimate returns under another policy ***π***.
- **Objective:** Estimate the expected return $v_\pi(s)$ using episodes generated by behavior policy b.

$$v_\pi(s) \approx \mathbb{E}_b[\rho_{1:T} \cdot G_t \mid S_t = s]$$

$$\rho_{1:T} = \prod_{t=1}^{T} \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}$$

$\rho_{1:T}$ is the cumulative importance sampling ratio

Application Example: Off-Policy Evaluation

- Suppose we have episodes collected under a random behavior policy ***b***.
- Using importance sampling, we evaluate the target policy ***π***.

$$v_\pi(s) \approx \frac{\sum_{i=1}^{N} \rho_{1:T}^{(i)} G_t^{(i)}}{\sum_{i=1}^{N} \rho_{1:T}^{(i)}}$$

Temporal Difference (TD) Learning

Introduction to TD Learning

Temporal-Difference (TD) methods update value estimates **directly** from experience.

- **Key Features:**
  - TD is **model-free**, i.e. it does not require knowledge of the MDP's transition probabilities or rewards.
  - It can learn from **partial episodes** by using a process called ***bootstrapping***.
    i. Therefore usually **faster** to converge, but **Biased**
  - TD updates involve adjusting one estimate in the direction of another estimate.

TD vs MC

Objective: Estimate the value function online through experience under policy **π**.

- **Incremental every-visit Monte Carlo:**
  - Update the value ***V(S(t))*** to move closer to the actual return ***G(t)***:

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

- **TD(0) - The Basic TD Learning Algorithm:**
  - Update the value ***V(S(t))*** to approximate the expected return ***R(t+1)+γV(S(t)+1)***:

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

  - TD Target:

$$\text{TD Target} = R_{t+1} + \gamma V(S_{t+1})$$

  - TD Error ***δ(t)***:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

TD vs MC Learning

Temporal-Difference (TD) Learning:

- TD can begin learning **without waiting the end of the episode**.
- It **updates values online**, adjusting after each step.
- It can learn from partial, **ongoing sequences**, and operates effectively in **continuing environments** that do not necessarily terminate.

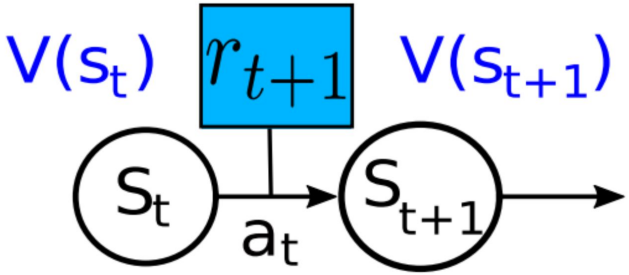
Monte Carlo (MC) Learning:

- MC **requires the full episode to finish** before calculating returns.
- It depends on **complete sequences**, so it is suited to **episodic environments** that have clear terminations.

TD Learning

- ***δ(t)*** measures the error between ***V (s(t))*** and the value it should have given ***r(t+1) + γV (s(t+1))***

- If ***δ(t)*** > 0, ***V(s(t))*** is under-evaluated, otherwise it is over-evaluated
- ***V(s(t))*** ← ***V(s(t)) + αδ(t)*** should decrease the error (value propagation)



$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

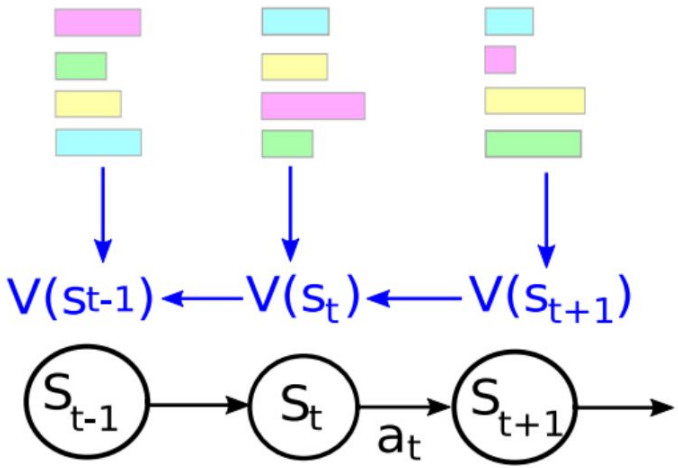
TD Learning

Update Rule

$$E_{k+1}(s) = E_k(s) + \alpha[r_{k+1} - E_k(s)] \qquad (1)$$

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \qquad (2)$$

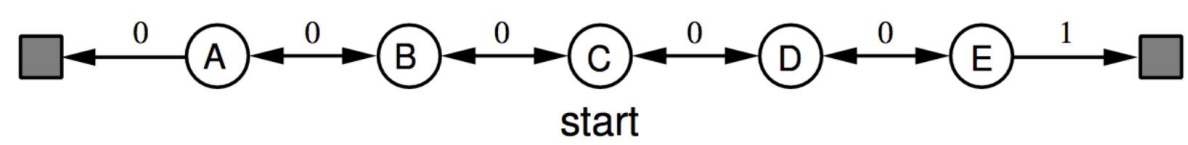
$$V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)] \qquad (3)$$



$$V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

- Combines two estimation processes:
  - incremental estimation (1)
  - value propagation from $V(s_{t+1})$ to $V(s_t)$ (2)

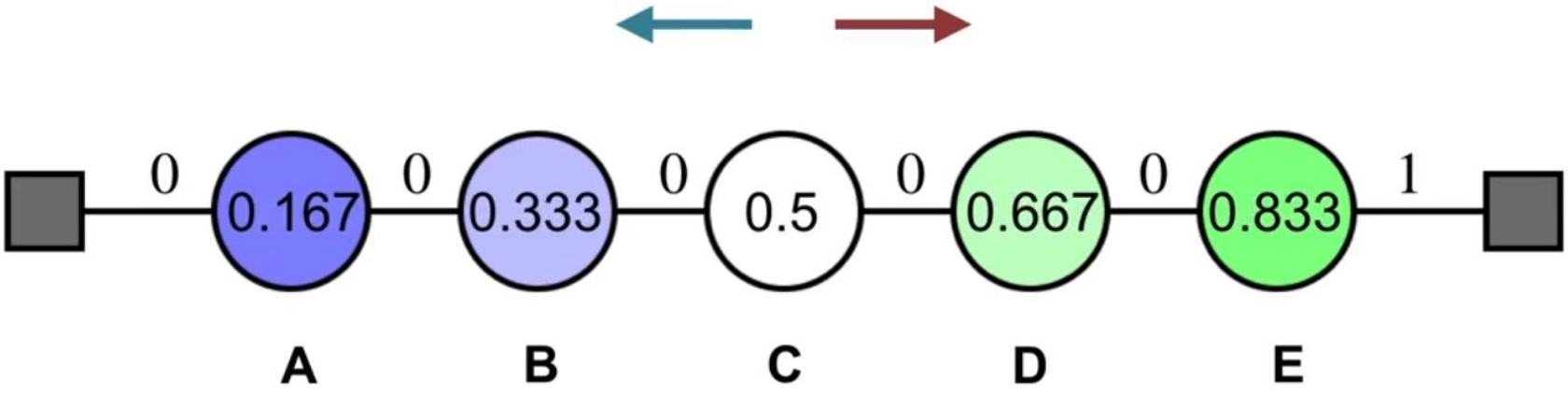
Example: Random Walk



$$\pi(\,.\,|\,s) = 1/2 \quad \forall s \in \mathcal{S} \qquad\qquad \gamma = 1$$

Example: Random Walk

- **True/Target Values**

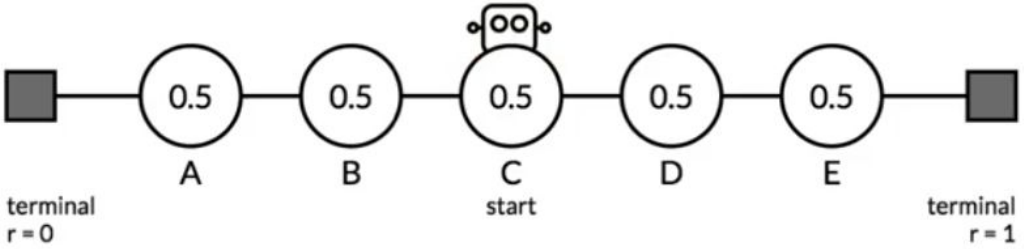


Example: Random Walk

Initialize the state-values for MC and TD methods

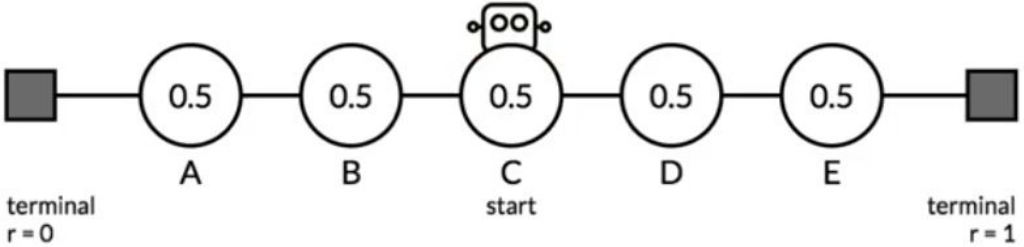
$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

TD learning



$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

MC Learning

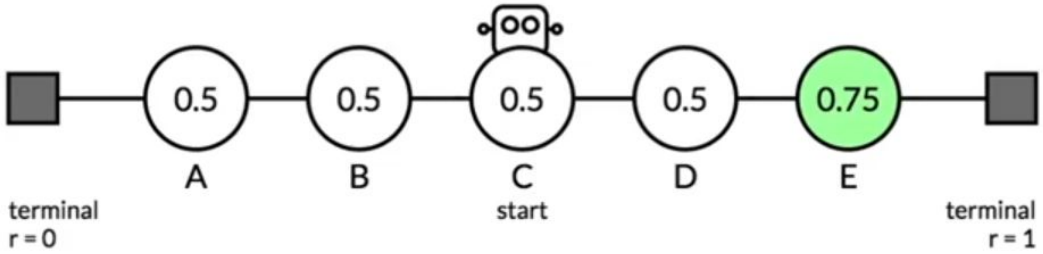


Example: Random Walk

After episode 1

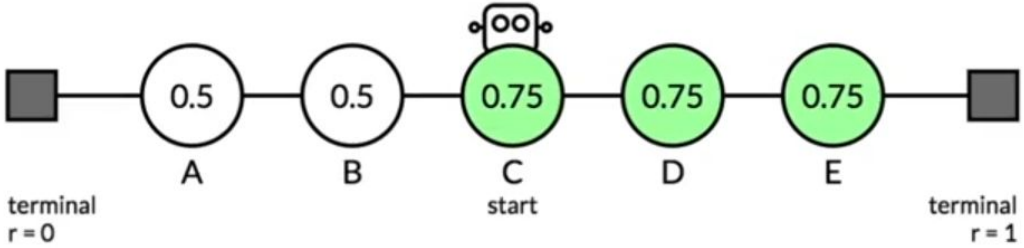
$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

TD learning



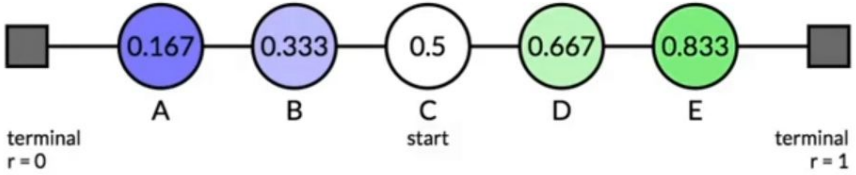
$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

MC Learning

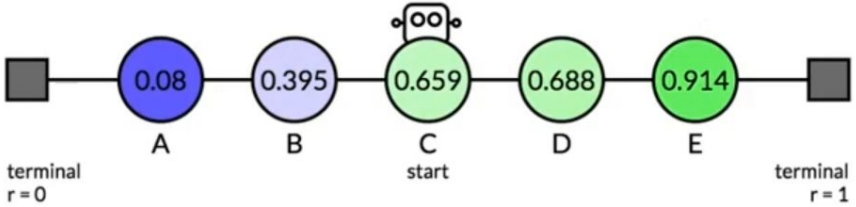


Example: Random Walk

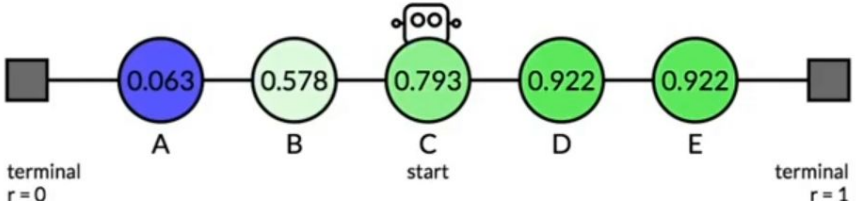
After multiple episodes



Updates using TD Learning

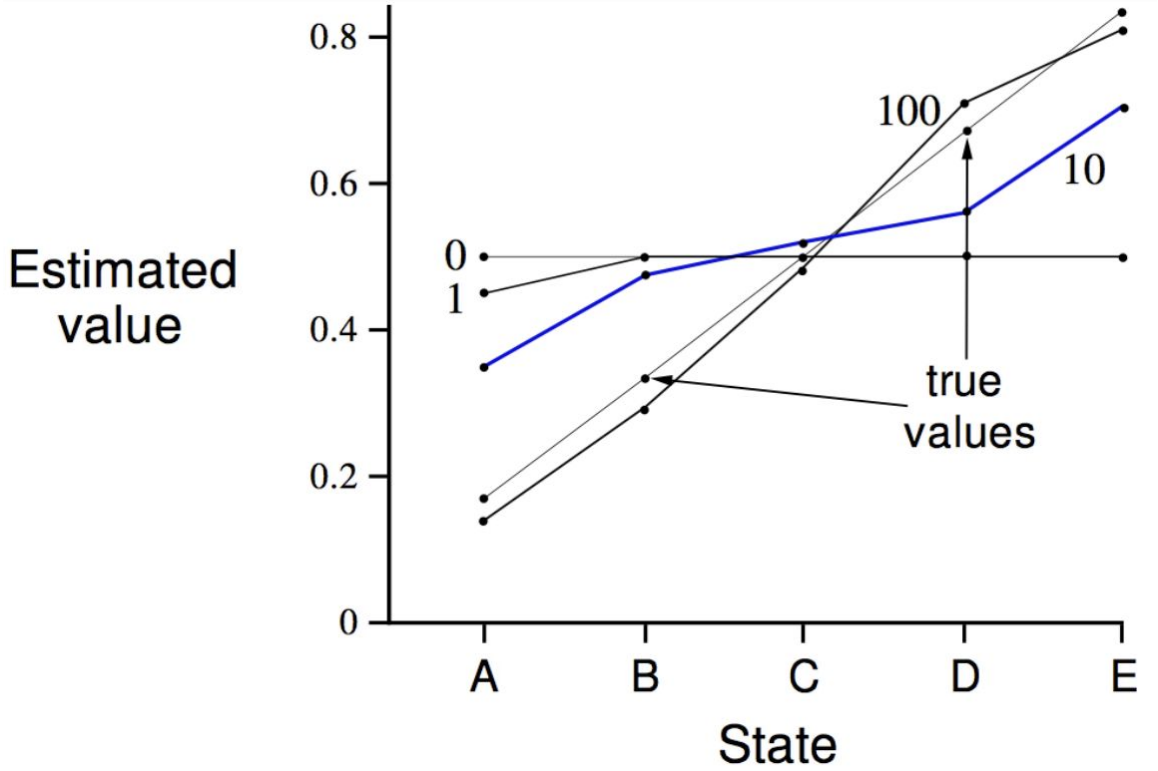


Updates using Monte Carlo



Example: Random Walk

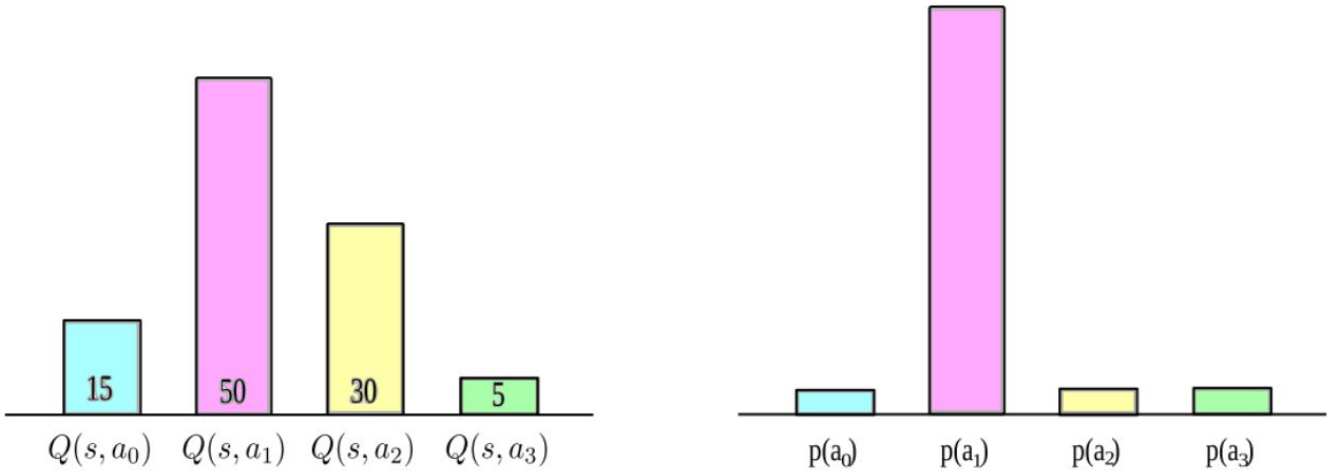
After 100 episodes



Exploration Tools

Exploration Tools

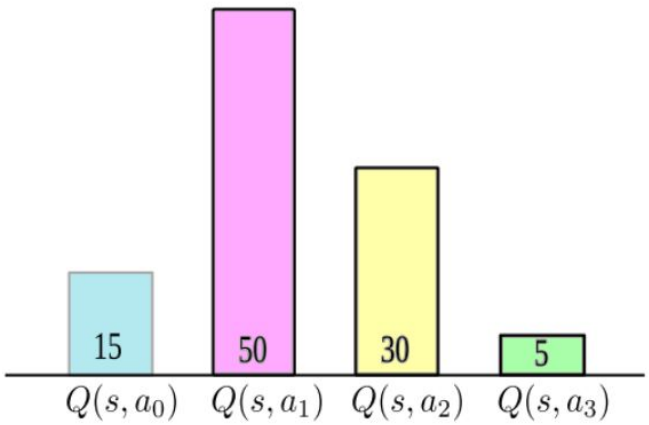
Epsilon-greedy exploration

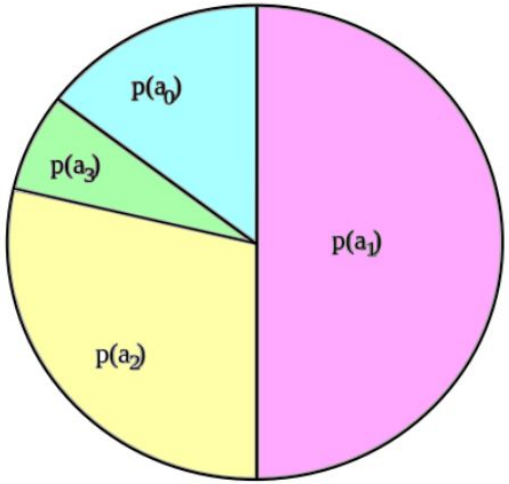


- Choose the best action with a high probability, other actions at random with low probability
- Same properties as random search
- Every state-action pair will be enough visited under an infinite horizon
- Useful for convergence proofs

Exploration Tools

Roulette wheel





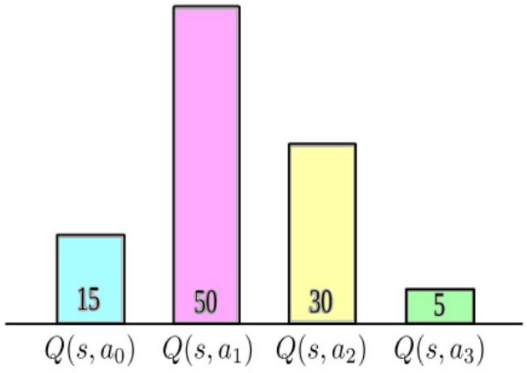
$$p(a_i) = \frac{Q(s, a_i)}{\sum_j Q(s, a_j)}$$

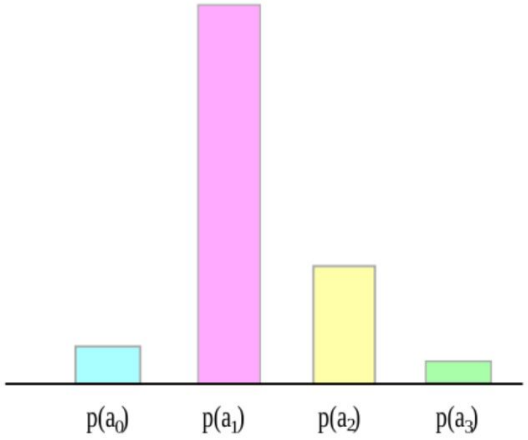
- The probability of choosing each action is proportional to its value

Exploration Tools

Softmax Exploration

$$p(a_i) = \frac{e^{\frac{Q(s,a_i)}{\beta}}}{\sum_j e^{\frac{Q(s,a_j)}{\beta}}}$$





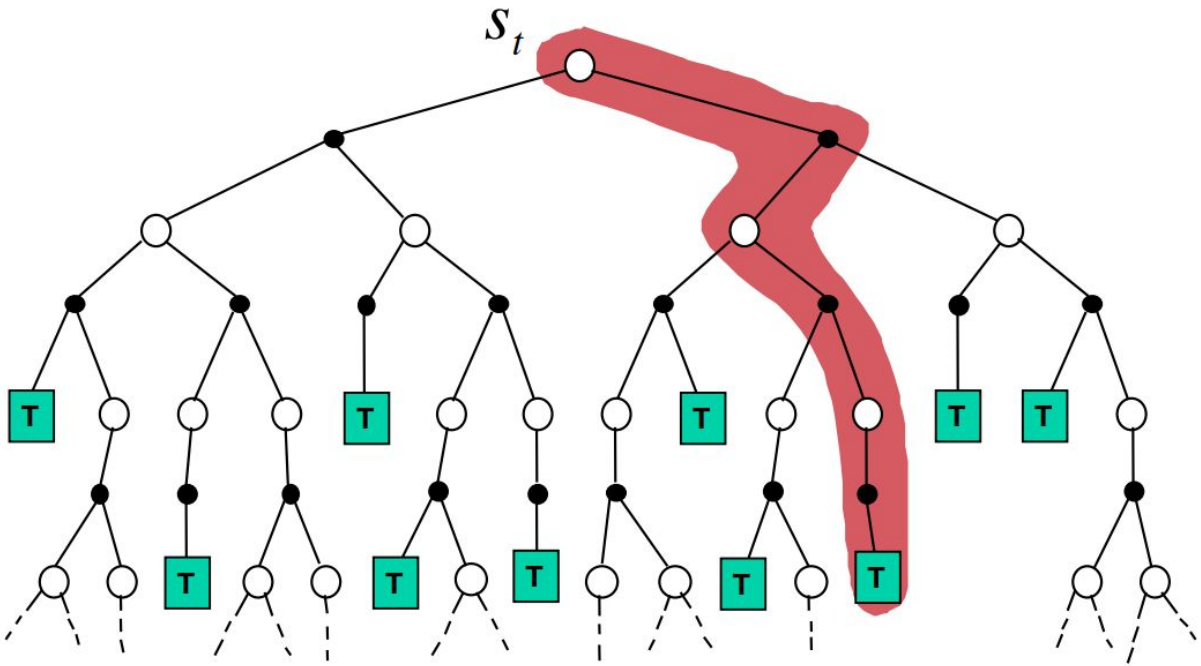
- The parameter β is called the temperature
- If $\beta \to \infty$, all actions have the same probability $\to$ random choice
- If $\beta \to 0$, increase contrast between values
- More used in computational neurosciences
- In machine learning RL, one often uses $\tau = \frac{1}{\beta}$, i.e. $p(a_i) = \frac{e^{\tau Q(s,a_i)}}{\sum_j e^{\tau Q(s,a_j)}}$
- Meta-learning: tune β dynamically (exploration/exploitation)

Backup

Backup

Monte Carlo

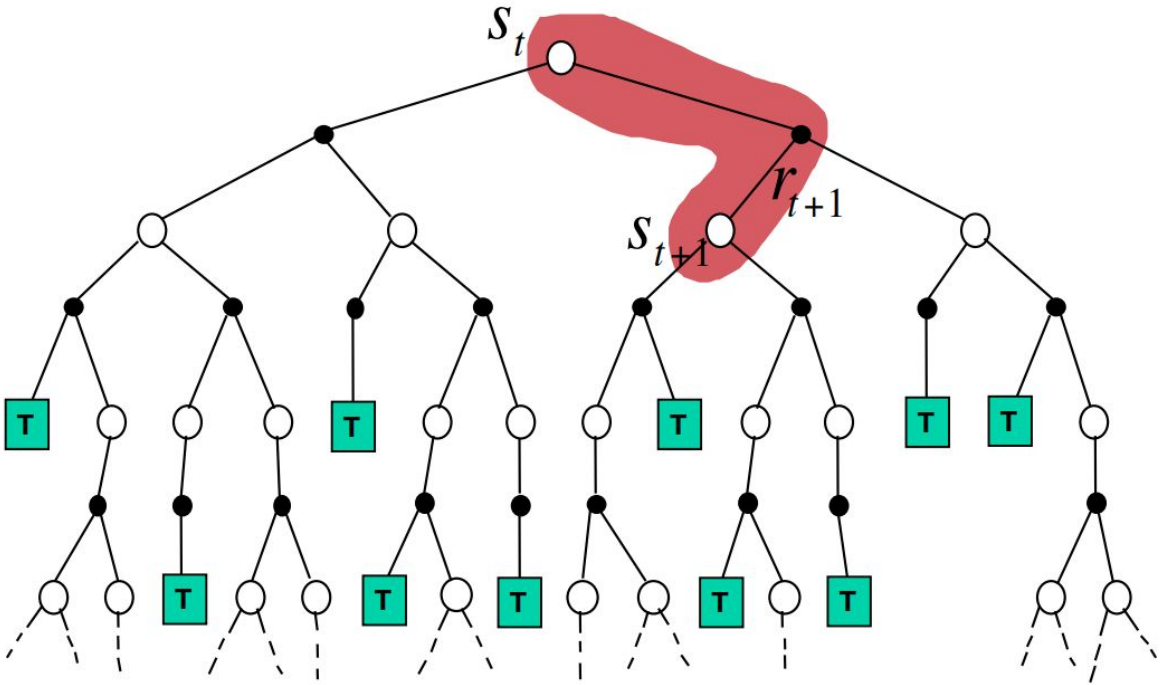
$$V(S_t) \leftarrow V(S_t) + \alpha\,(G_t - V(S_t))$$



Backup

Temporal Difference

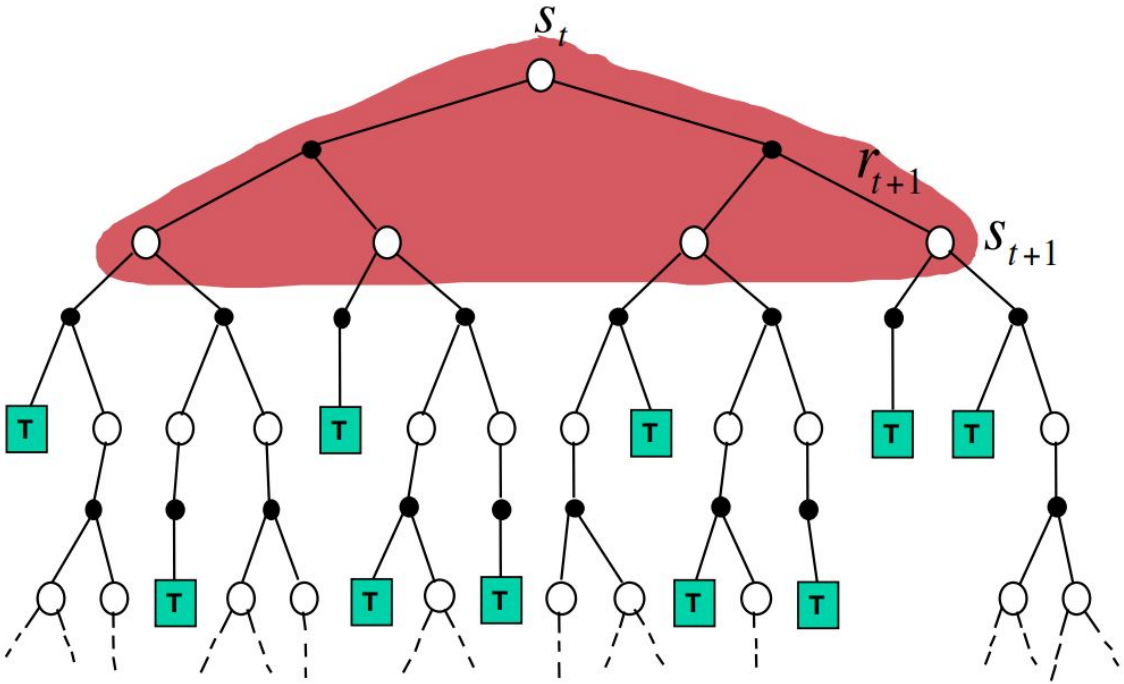
$$V(S_t) \leftarrow V(S_t) + \alpha\left(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right)$$



Backup

Dynamic Programming

$$V(S_t) \leftarrow \mathbb{E}_\pi \left[R_{t+1} + \gamma V(S_{t+1})\right]$$



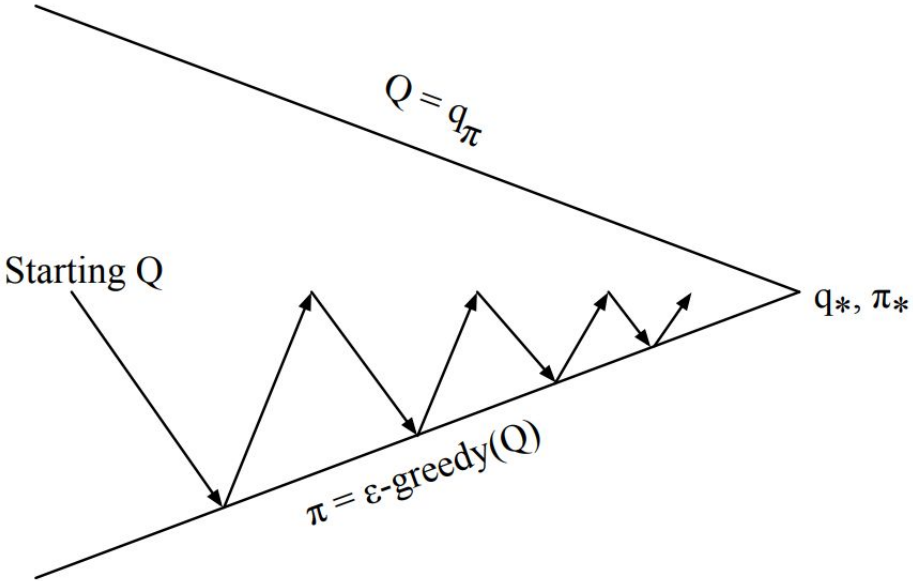
TD for Control

Sarsa

On-policy TD Control

Each time step:

- **Policy Evaluation:** Using Sarsa, approximate $Q \approx q_\pi$
- **Policy Improvement:** Apply ϵ-greedy policy improvement.



Sarsa

- Reminder (TD): $V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$
- SARSA: For each observed $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$:
  $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$
- Policy: perform exploration (e.g. ϵ-*greedy*)
- One must know the action a_{t+1}, thus constrains exploration
- On-policy method: more complex convergence proof

Sarsa

On-policy TD Control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Q-Learning

- For each observed $(s_t, a_t, r_{t+1}, s_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_{a \in A} Q(s_{t+1}, a) - Q(s_t, a_t)]$$

- $\max_{a \in A} Q(s_{t+1}, a)$ instead of $Q(s_{t+1}, a_{t+1})$
- Off-policy method: no more need to know a_{t+1}
- Policy: perform exploration (e.g. ϵ-*greedy*)
- Convergence proven given infinite exploration

Q-Learning

Off-policy TD Control

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $\boxed{Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]}$
    $S \leftarrow S'$
  until S is terminal

Q-Learning

Checking with Bellman Equations

Sarsa

Uses Bellman equation

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big)$$

$$q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a)\left(r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a')\right)$$

Q-Learning

Uses Bellman optimality equation

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\big)$$

$$q_*(s, a) = \sum_{s', r} p(s', r \mid s, a)\big(r + \gamma \max_{a'} q_\pi(s', a')\big)$$

Q-Learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_a Q(s',a) - Q(s,a)\right]$$

Game Board:

Q Table: γ = 0.95

	0 0 0 1 0 0	0 0 0 0 1 0	0 0 0 0 0 1	1 0 0 0 0 0	0 1 0 0 0 0	0 0 1 0 0 0
↑	0.2	0.3	1.0	-0.22	-0.3	0.0
↓	-0.5	-0.4	-0.2	-0.04	-0.02	0.0
→	0.21	**0.95**	-0.3	0.5	1.0	0.0
←	-0.6	-0.1	-0.1	-0.31	-0.01	0.0

Current state (*s*): **0 0 0 / 0 1 0**

Selected action (*a*): →

Reward (*r*): **0**

Next state (*s'*): **0 0 0 / 0 0 1**

max Q(s'): **1.0**

New Q(s,a) = r + γ * maxQ(s') = 0 + 0.95 * 1 = **0.95**

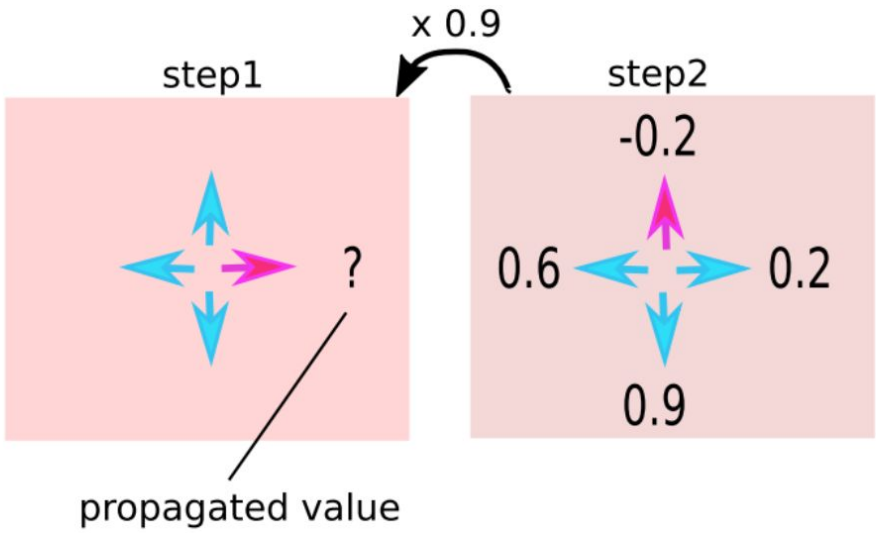
Why Q-Learning is Off-Policy?

Target policy: π^* $= \underset{a}{\operatorname{argmax}}\, Q(s, a)$

Behavior policy: π

$$\pi_* \neq \pi$$

Sarsa vs Q-Learning



- Consider an agent taking the two pink actions
- With Q-LEARNING, the propagated value ? is $\gamma \operatorname{argmax}_a Q(s_{t+1}, a)$, thus $0.9 \times 0.9 = 0.81$
- With SARSA, it is $\gamma Q(s_{t+1}, a_{t+1})$, thus $0.9 \times -0.2 = -0.18$

Expected Sarsa

Sarsa

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big)$$

Bellman Equation

$$q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a)\left(r + \gamma \boxed{\sum_{a'} \pi(a' \mid s') q_\pi(s', a')}\right)$$

Expected action-value

Expected Sarsa

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big(R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t)\big)$$

Note on Expected Sarsa

- **Expected Sarsa**

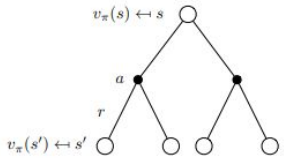
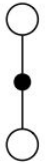
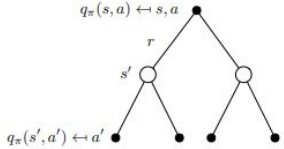
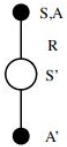
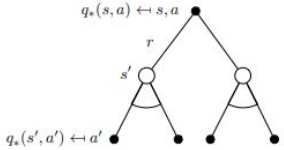
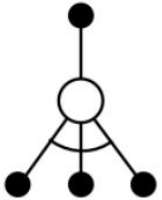
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t)\Big)$$

$$\cancel{A_{t+1}} \sim \pi(a' \mid S_{t+1}) \neq b(a' \mid S_{t+1})$$

- **Expected Sarsa** can be considered as **off-policy** learning too!

Summary

Summary

	Full Backup (DP)	*Sample Backup (TD)*
Bellman Expectation Equation for $v_\pi(s)$	 Iterative Policy Evaluation	 TD Learning
Bellman Expectation Equation for $q_\pi(s, a)$	 Q-Policy Iteration	 Sarsa
Bellman Optimality Equation for $q_*(s, a)$	 Q-Value Iteration	 Q-Learning

Summary

Full Backup (DP)	*Sample Backup (TD)*
Iterative Policy Evaluation $V(s) \leftarrow \mathbb{E}\left[R + \gamma V(S') \mid s\right]$	TD Learning $V(S) \overset{\alpha}{\leftarrow} R + \gamma V(S')$
Q-Policy Iteration $Q(s,a) \leftarrow \mathbb{E}\left[R + \gamma Q(S',A') \mid s,a\right]$	Sarsa $Q(S,A) \overset{\alpha}{\leftarrow} R + \gamma Q(S',A')$
Q-Value Iteration $Q(s,a) \leftarrow \mathbb{E}\left[R + \gamma \max_{a' \in \mathcal{A}} Q(S',a') \mid s,a\right]$	Q-Learning $Q(S,A) \overset{\alpha}{\leftarrow} R + \gamma \max_{a' \in \mathcal{A}} Q(S',a')$

$$\boxed{x \overset{\alpha}{\leftarrow} y \equiv x \leftarrow x + \alpha(y - x)}$$